

Autodesk® Scaleform®

DrawText API

Scaleform 4.4 DrawText API . API
Gfx::Movie C++
.

: Artem Bolgar
:
: 2011 4 22

Copyright Notice

Autodesk® Scaleform® 4.4

© 2014 Autodesk, Inc. All rights reserved. Except as otherwise permitted by Autodesk, Inc., this publication, or parts thereof, may not be reproduced in any form, by any method, for any purpose.

Certain materials included in this publication are reprinted with the permission of the copyright holder.

The following are registered trademarks or trademarks of Autodesk, Inc., and/or its subsidiaries and/or affiliates in the USA and other countries: 123D, 3ds Max, Algor, Alias, AliasStudio, ATC, AutoCAD LT, AutoCAD, Autodesk, the Autodesk logo, Autodesk 123D, Autodesk CAM 360, Autodesk Homestyler, Autodesk Inventor, Autodesk MapGuide, Autodesk Streamline, AutoLISP, AutoSketch, AutoSnap, AutoTrack, Backburner, Backdraft, Beast, BIM 360, Burn, Buzzsaw, CADmep, CAiCE, CAMduct, CFdesign, Civil 3D, Cleaner, Combustion, Communication Specification, Configurator 360™, Constructware, Content Explorer, Creative Bridge, Dancing Baby (image), DesignCenter, DesignKids, DesignStudio, Discreet, DWF, DWG, DWG (design/logo), DWG Extreme, DWG TrueConvert, DWG TrueView, DWGX, DXF, Ecotect, ESTmep, Evolver, FABmep, Face Robot, FBX, Fempro, Fire, Flame, Flare, Flint, FMDesktop, ForceEffect, FormIt, Freewheel, Fusion 360, Glue, Green Building Studio, Heidi, Homestyler, HumanIK, i-drop, ImageModeler, Incinerator, Inferno, InfraWorks, InfraWorks 360, Instructables, Instructables (stylized robot design/logo), Inventor, Inventor HSM, Inventor LT, Kynapse, Kynogon, LandXplorer, Lustre, MatchMover, Maya, Maya LT, Mechanical Desktop, MIMI, Mockup 360, Moldflow Plastics Advisers, Moldflow Plastics Insight, Moldflow, Moondust, MotionBuilder, Movimento, MPA (design/logo), MPA, MPI (design/logo), MPX (design/logo), MPX, Mudbox, Navisworks, ObjectARX, ObjectDBX, Opticore, Pipeplus, Pixlr, Pixlr-omatic, Productstream, Publisher 360, RasterDWG, RealDWG, ReCap, ReCap 360, Remote, Revit LT, Revit, RiverCAD, Robot, Scaleform, Showcase, Showcase 360 ShowMotion, Sim 360, SketchBook, Smoke, Socialcam, Softimage, Sparks, SteeringWheels, Stitcher, Stone, StormNET, TinkerBox, ToolClip, Topobase, Toxik, TrustedDWG, T-Splines, ViewCube, Visual LISP, Visual, VRED, Wire, Wiretap, WiretapCentral, XSI.

All other brand names, product names or trademarks belong to their respective holders.

Disclaimer

THIS PUBLICATION AND THE INFORMATION CONTAINED HEREIN IS MADE AVAILABLE BY AUTODESK, INC. "AS IS." AUTODESK, INC. DISCLAIMS ALL WARRANTIES, EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE REGARDING THESE MATERIALS.

:

DrawText API Autodesk Scaleform Corporation 6305 Ivy Lane, Suite 310 Greenbelt, MD 20770, USA www.scaleform.com info@scaleform.com (301) 446-3200 (301) 446-3199
--

목차

- 1 들어가기 1
 - 1.1 Gfx::DrawTextManager.....2
 - 1.1.1. DrawText 3
 - 1.1.2. 5
 - 1.1.3. 5
 - 1.2 Gfx::DrawText6
 - 1.2.1 (set), (get) 7
 - 1.2.2 HTML (set), (get) 7
 - 1.2.3 8
 - 1.2.4 9
 - 1.2.5 9
- 2 DrawText 예제코드 요약 11

1 들어가기

Scaleform SDK
UI
Scaleform
(glyph)
HTML
SWF/GFX
Scaleform
TextFormat
TextFormat
API
C++ API
Scaleform
UI
C++
Scaleform
3D
HUD
Scaleform3
C++ DrawText API
Scaleform
Scaleform
Render::Renderer
API
GFX::Movie
DrawText API
2
SWF/GFX
SWF
GFX::MovieDef
(GFX::Loader::CreateMovie)
GFX::DrawTextManager
GFX::MovieDef
C++
GFX::DrawTextManager
GFX::DrawText

- *GfX::DrawTextManager* - DrawText . , , ,
 , /
- *GfX::DrawText* – .
 . HTML
 . SetColor, SetFont, SetFontStyle
 .

1.1 *GfX::DrawTextManager*

DrawTextManager DrawText . , , ,
 , .

DrawTextManager .

1. DrawTextManager();

.
 (SetFontProvider)

:

Ptr<DrawTextManager> pdm = *new DrawTextManager();

2. DrawTextManager(MovieDef* pmovieDef);

MovieDef . DrawTextManager MovieDef
 (,). MovieDef
 DrawTextManager (DrawTextManager
 DrawText).

:

Ptr<GfX::MovieDef> pmovieDef = *mLoader.CreateMovie("drawtext_fonts.swf",
 Loader::LoadAll);

Pt6905B07Sjfa0002Tj2D0dGFr)UjHm0CPar/amsIw/(Dc12)Tp0>FBJBB0/ITf00.002Tj00.602240c0.55o0J2B7Q29c96

```

        float          FontSize;
        String         FontName;
        bool           Underline;
        bool           Multiline;
        bool           WordWrap;
};

TextColor :           ,

HAlignment:           . (DrawText::Align_Left (default), DrawText::Align_Right,
DrawText::Align_Center, DrawText::Align_Justify.)

VAlignment :           ,
(DrawText::VAlign_Top (default), DrawText::VAlign_Bottom, DrawText::VAlign_Center)

FontStyle :           ,           (DrawText::Normal, DrawText::Bold,
DrawText::Italic, DrawText::BoldItalic (DrawText::ItalicBold           )).

FontSize:           ,

FontName:           , "Arial", "Times New Roman"           . "Bold", "Italic"
        FontStyle           .

Underline:

Multiline:           ,

WordWrap:           (wordwrapping)           ,

ptxtParams           DrawTextManager

        .

void SetDefaultTextParams(const TextParams& params);
const TextParams& GetDefaultTextParams() const;

        DrawTextManager           ( , CreateHtmlText)           CreateText           ,
HTML           DrawText           .

// Creates and initialize a DrawText object using specified HTML.

DrawText* CreateHtmlText(const char* putf8Str, const RectF& viewRect,
        const TextParams* ptxtParams = NULL, unsigned depth = ~0u);
DrawText* CreateHtmlText(const wchar_t* pwstr, const RectF& viewRect,
        const TextParams* ptxtParams = NULL, unsigned depth = ~0u);

```



```

DrawText* CreateHtmlText(const String& str, const RectF& viewRect,
                        const TextParams* ptxtParams = NULL, unsigned depth = ~0u);

:

// Creation of DrawText object using plain text with parameters.
String str("String No 2");
DrawTextManager::TextParams params;
params.FontName    = "Symbol";
params.FontSize    = 30;
params.FontStyle   = DrawText::Italic;
params.Multiline   = false;
params.HAlignment  = DrawText::Align_Right;
params.VAlignment  = DrawText::VAlign_Bottom;

Ptr<DrawText> ptxt;
ptxt = *pdm->CreateText(str, GRectF(20, 300, 400, 700), &params);

// Creation of DrawText object from HTML.
Ptr<DrawText> ptxt;
ptxt = *pdm->CreateHtmlText("<p><FONT size='20'>AB
                           <b>singleline</b><i> CD</i>0",
                           RectF(20, 300, 400, 700));

```

1 8b \$21 sT T32 1 Tf 0 Tc 0 Tw24. 0 Td TC212 1 Tf 0303 0 Td 0A00AA E

```

GetTextExtent
TextParams
GetHtmlTextExtent    HTML
TextParams

    'width'

    .

    ptxtParams

    . (SetDefaultTextParams /
GetDefaultTextParams    ) HTML    ptxtParams
    ,    , HTML    ptxtParams
    .
    :

    // Plain text extents
    String str("String No 2");
    DrawTextManager::TextParams params;
    params.FontName    = "Symbol";
    params.FontSize    = 30;
    params.FontStyle    = DrawText::Italic;
    params.Multiline    = false;
    params.HAlignment    = DrawText::Align_Right;
    params.VAlignment    = DrawText::VAlign_Bottom;

    SizeF sz = pdm->GetTextExtent(str, 0, params);

    params.WordWrap    = true;
    params.Multiline    = true;
    sz = pdm->GetTextExtent(str, 120, params);

    // HTML text extents
    const wchar_t* html =
        L"<p><FONT size='20'>AB <b>singleline</b><i> CD</i>0";
    sz = pdm->GetHtmlTextExtent(html);

    sz = pdm->GetHtmlTextExtent(html, 150);

```

1.2 Gfx::DrawText

DrawText , HTML , , .

1.2.1 일반 텍스트 설정(set), 얻기(get) 메소드

SetText UTF-8, UCS-2, Scaleform::String .
 'lengthInBytes' UTF-8 , 'lengthInChars' wide
 , UTF-8 UCS-2 NULL
 .
 .

```
void SetText(const char* putf8Str, UPInt lengthInBytes = UPInt(-1));
void SetText(const wchar_t* pstr, UPInt lengthInChars = UPInt(-1));
void SetText(const String& str);
```

GetText UTF-8 .
 , HTML , HTML .

```
String GetText() const;
```

1.2.2 HTML 텍스트 설정(set), 얻기(get) 메소드

SetHtmlText HTML UTF-8, UCS-2, Scaleform::String .
 'lengthInBytes' UTF-8 , 'lengthInChars'
 wide , UTF-8 UCS-2
 NULL .

```
void SetHtmlText(const char* putf8Str, UPInt lengthInBytes = UPInt(-1));
void SetHtmlText(const wchar_t* pstr, UPInt lengthInChars = UPInt(-1));
void SetHtmlText(const String& str);
```

GetHtmlText HTML . SetColor,
 SetFont , HTML .

```
String GetHtmlText() const;
```

1.2.3 텍스트 포맷 설정 메소드

```
void SetColor(Color c, UPInt startPos = 0, UPInt endPos = UPInt(-1));
                                [startPos..endPos]                (R, G, B, A)
'startPos'    'endPos'

void SetFont (const char* pfontName, UPInt startPos = 0, UPInt endPos = UPInt(-1))
                                [startPos..endPos]                . 'startPos'    'endPos'

void SetFontSize(float fontSize, UPInt startPos = 0, UPInt endPos = UPInt(-1))
                                [startPos..endPos]                . 'startPos'
'endPos'

void SetFontStyle(FontStyle, UPInt startPos = 0, UPInt endPos = UPInt(-1));
enum FontStyle
{
    Normal,
    Bold,
    Italic,
    BoldItalic,
    ItalicBold = BoldItalic
};
                                [startPos..endPos]                . 'startPos'    'endPos'

void SetUnderline(bool underline, UPInt startPos = 0, UPInt endPos = UPInt(-1))
                                [startPos..endPos]                . 'startPos'    'endPos'

void SetMultiline(bool multiline);
                                (true)        (false)

bool IsMultiline() const;
                                true        false

void SetWordWrap(bool wordWrap);

bool Wrap() const;
```

1.2.4 텍스트 정렬 메소드

Alignment VAlignment

```
enum Alignment
{
    Align_Left,
    Align_Default = Align_Left,
    Align_Right,
    Align_Center,
    Align_Justify
};
enum VAlignment
{
    VAlign_Top,
    VAlign_Default = VAlign_Top,
    VAlign_Center,
    VAlign_Bottom
};
```

```
void SetAlignment(Alignment);
                (right, left, center).

Alignment  GetAlignment() const;
                (right, left, center).

void SetVAlignment(VAlignment);
                (top, bottom, center).

VAlignment GetVAlignment() const;
                (top, bottom, center).
```

1.2.5 텍스트 위치와 방위 변경

DrawText

```
void SetRect(const RectF& viewRect);
                ,

RectF  GetRect() const;
                ,

void SetMatrix(const Matrix& matrix);
```

```
const Matrix GetMatrix() const;
```

```
void SetCxform(const Cxform& cx);
```

```
const Cxform& GetCxform() const;
```

2 DrawText 예제 코드 요약

Scaleform 4.1 DrawText API . DrawText API

Scaleform SDK

DrawText API

FxPlayerApp

FxPlayerApp FxPlayerAppBase

FxPlayerApp OnInit DrawText . OnUpdateFrame

```

class FxPlayerApp : public FxPlayerAppBase
{
public:
    FxPlayerApp();

    virtual bool      OnInit(Platform::ViewConfig& config);
    virtual void      OnUpdateFrame(bool needRepaint);

    Ptr<GFx::DrawTextManager>    pDm1, pDm2;
    Ptr<GFx::DrawText>          ptxt11, ptxt12, ptxt21, ptxt22, ptxtImg,
                                pblurTxt, pglowTxt, pdropShTxt, pblurglowTxt;

    float    Angle;
    UInt32   Color;
};

```

DrawText DrawTextManager 2

방법 1

DrawTextManager

Gfx::Loader

```
pDm1 = *new DrawTextManager(&mLoader);
```

Gfx::FontProvider

FontProvider FxPlayerAppBase::OnInit

```
pDm1->SetFontProvider(mLoader.GetFontProvider());
```

API

DrawText

HTML

Scaleform::String

```
// Create text using plain text and default text parameters.
```

```
DrawTextManager::TextParams defParams =
```

```
    pDm1->GetDefaultTextParams();
```

```
    defParams.TextColor = Color(0xF0, 0, 0, 0xFF); // red, alpha = 255
```

```
    defParams.FontName = "Arial";
```

```
    defParams.FontSize = 16;
```

```
    pDm1->SetDefaultTextParams(defParams);
```

CreateText

SetDefaultTextParams

```
ptxt11 = *pDm1->CreateText ("Plain text, red, Arial,  
16pts",RectF(20, 20, 500, 400));
```

DrawTextManager

GetTextExtent

. DrawText

```
// Create text with using String and TextParams
```

```
String str(
```

```
    "Scaleform Gfx is a light-weight high-performance rich media vector  
    graphics and user interface (UI) engine.");
```

```
Gfx::DrawTextManager::TextParams params;
```

```
params.FontName = "Arial";
```

```
params.FontSize = 14;
```

```
params.FontStyle = DrawText::Italic;
```

```
params.Multiline = true;
```

```
params.WordWrap = true;
```

```
params.HAlignment = DrawText::Align_Justify;
```

```
sz = pDm1->GetTextExtent(str, 200, &params);
```



```
ptxt12 = *pDm1->CreateText(str, RectF(200, 300, sz), &params);
ptxt12->SetColor(Render::Color(0, 0, 255, 130), 0, 1);
```

DrawText

DrawText

```
DrawText DisplayHandle RenderThread .
Render::Size<unsigned> viewSize = GetViewSize();
Render::Viewport dmViewport(viewSize.Width, viewSize.Height,
    int(viewSize.Width * GetSafeArea().Width),
    int(viewSize.Height * GetSafeArea().Height),
    int(viewSize.Width - 2 * viewSize.Width * GetSafeArea().Width),
    int(viewSize.Height - 2 * viewSize.Height * GetSafeArea().Height));
pDm1->SetViewport(dmViewport);
pDm1->Capture();
pRenderThread->AddDisplayHandle(pDm1->GetDisplayHandle(),
    FxRenderThread::DHCAT_Overlay, false);
```

방법 2

DrawTextManger

GfX::MovieDef

. MovieDef

SWF

. Scaleform

Scaleform Integration Tutorial

```
Ptr<MovieDef> pmovieDef = *mLoader.CreateMovie("drawtext_fonts.swf",
    Loader::LoadAll);
pDm2 = *new DrawTextManager(pmovieDef);
```

HTML

DrawTextManager::GetHtmlExtent

```
// Create HTML text, using fonts from GfX::MovieDef
const wchar_t* html = L"<P>o123 <FONT FACE=\"Times New Roman\" SIZE
    =\"140\">\"L\"A<b><i><FONT
    COLOR='#3484AA'>о</FONT></i>рак</b>адабрА!</FONT></P>"
    L"<P><FONT FACE='Arial Unicode MS'>Hànyǔ; 华语/ </FONT></P>"
    L"<P><FONT FACE='Batang'> / </FONT></P>"
    L"<P><FONT FACE='Symbol'>Privet!</FONT></P>";

GfX::DrawTextManager::TextParams defParams2 = pDm2->GetDefaultTextParams();
defParams2.TextColor = Color(0xF0,0,0,0xFF); //red,alpha = 255
defParams2.Multiline = true;
defParams2.WordWrap = false;
SizeF htmlSz = pDm2->GetHtmlTextExtent(HtmlText, 0, &defParams2);
```

```

    ptxt22 = *pDm2->CreateHtmlText(HtmlText, RectF(00, 100, htmlSz),
&defParams2);

```

```

(
)

```

```

SizeF sz;
sz = pDm2->GetTextExtent("Animated");
ptxt21 = *pDm2->CreateText("Animated", RectF(600, 400, sz));
ptxt21->SetColor(Render::Color(0, 0, 255, 255));
Angle = 0;

```

```

:

```

```

SizeF sz;
Ptr<Scaleform::DrawText> pglowTxt
pglowTxt = *pDm2->CreateText("Glow", RectF(800, 350, SizeF(200, 220)));
pglowTxt->SetColor(Render::Color(120, 30, 192, 255));
pglowTxt->SetFontSize(72);
Scaleform::DrawText::Filter glowF(Scaleform::DrawText::Filter_Glow);
glowF.Glow.BlurX = glowF.Glow.BlurY = 2;
glowF.Glow.Strength = 1000;
glowF.Glow.Color = Render::Color(0, 0, 0, 255).ToColor32();
pglowTxt->SetFilters(&glowF);

```

```

    DrawText

```

```

    DrawText

```

```

    DrawText

```

```

DisplayHandle    RenderThread

```

```

.

```

```

pDm2->SetViewport(dmViewport);
pDm2->Capture();
pRenderThread->AddDisplayHandle(pDm2->GetDisplayHandle(),
                                FxRenderThread::DHCAT_Overlay, false);

```

```

DrawText

```

```

DrawText    SetMatrix    SetCxform

```

```

    FxPlayerApp::OnUpdateFrame

```

```

void FxPlayerApp::OnUpdateFrame( bool needRepaint )
{
    DrawText::Matrix txt21matrix;
    Angle += 1;
    RectF r = ptxt21->GetRect();
    txt21matrix.AppendTranslation(-r.x1, -r.y1);
    txt21matrix.AppendRotation(Angle*3.14159f / 180.f);
    txt21matrix.AppendScaling(2);
    txt21matrix.AppendTranslation(r.x1, r.y1);
    ptxt21->SetMatrix(txt21matrix);
}

```

```
pDm2->Capture();
```

```
FxPlayerAppBase::OnUpdateFrame(needRepaint);
```

```
}
```

```
: Bin/Samples drawtext_fonts.swf
```

```
(Visual Studio 2008 Projects/Win32/Msvc90/Samples/DrawText ).
```

```
:
```

